Chapter 7 on Instrumentation

Unit 7: Software for Instrumentation Application

(6 Hours / 6 Marks)

- Software is pervasive in electronic products such as televisions, video recorders, remote controls, microwave ovens, sewing machines, and cloth washers all have embedded microcontrollers.
- Software accounts for 50-75 percent of a microcontroller project.
- General methods to improve software are:
  - code generation
  - reliability
  - maintainability
  - correctness.

Types of software, Selection and Purchase

Types of software

- Software is found in many different types of systems such as real-time control, data processing systems like payroll, and graphical systems such as games and CAD. Software can be divided into following types:
- **System Software**
  - Operating system, System drivers, Firmware etc.
- **Programming Software**
  - Compiler, Debugger, Interpreter, Linker, Text editor etc.
- **Application Software**
  - Industrial / Business automation, User interface, Games / Simulating software, Database, Image editing, Auto CAD, word processor etc.

Compiler vs Interpreter: You Know It.

Processed to develop software

1. Algorithms

- List of instructions or recipes for action
- Algorithms describe the general actions to be taken and consequently are independent of the specific programming language.
- Algorithms have the greatest effect on the utility, success, and failure of software
- Data structures provide another way of designing the processing architecture.
- Make a habit of collecting algorithms for your future programming efforts; when crisis hits, there will be no time for research.
- Understand each algorithm, its limitations and its boundary conditions
- Jack Ganssle writes: “It’s ludicrous that we software people reinvent the wheel with every project… Wise programmers make an ongoing effort to built an arsenal of tools for current and future project…. Make an investment in collecting algorithms for future use. When a crisis hits there is no time to begin research.”

2. Languages

- Software has many applications within embedded systems such as Firmware, Peripheral interface and drivers, Operating system, User interface, Application programs etc.
- May use variety of languages
- Assembly language
- High level language: Basic, C, C++, Java etc.
- Difference, Merits and Demerits: You Know It.

3. Methods

- Whatever language you choose, your objective will be to reduce complexity and improve understanding of the software.
- Design architecture may be Structured or Object oriented or CASE (Computer aided software engineering)
- Structured designs have strategy before starting to code; small modules with clear operational flow, easy debugging and testing.
- OOP can help by incorporating data abstractions, information hiding and modularity to aid structured design.
- CASE tools provide blend of environment, tools and language.
- Tools available are compilers, disassemblers, debuggers, emulators, monitors and logic analysers.
- Operating system and software libraries ease the task.

4. Selection

- The selection of a particular language depends on management directives, the knowledge and expertise of the software team, hardware and available tools.
- Function and performance depends on the speed and data path width of processor, memory (RAM and ROM), architectural features such as coprocessors, peripherals (ADC, timers, PWM, interrupt handlers), I/O communications, power-down modes and the level of integration.
- Choice of language also depends on manufacturers having following questions :
  - Does the vendor provide reasonable documentation and support?
  - Does it provide toll-free telephone support and acknowledge application engineers?
  - Does it have liability support for life-critical systems?

5. Purchase

- Purchase the software after you have defined your software requirements and surveyed vendors for availability, reputation and experience.
- Some qualification of a vendor:
  - Acceptance testing
  - Review of vendor's quality assurance
  - Verification testing
  - Qualification report
- Furthermore, required documentations from a vendor:
  - Requirement specification
  - Interface specification
  - Test plans, procedures, results
  - Configuration management plan
  - Hazard analysis
- Don't buy cheap software tools just to save money! You will lose much more money in the long run from wasted time forced by delays and inadequacies of cheap tools.

Software Models and Their Limitations

Traditional software lifecycle / Waterfall Model

- Over the years many models have been proposed to deal with the problems of defining the critical activities and tying them together
- The first formally defined software life cycle model was the *waterfall model [Royce* 1970]
- The waterfall model is a software development model in which the results of one activity flowed sequentially into the next as seen as flowing steadily downwards (like a water) through different phases.
- Water cascades from one stage down to the next, in stately, lockstep, glorious order.
  - gravity only allows the waterfall to go downstream; it's hard to swim upstream
- The US Department of Defense contracts prescribed this model for software deliverables for many years, in DOD Standard 2167-A.

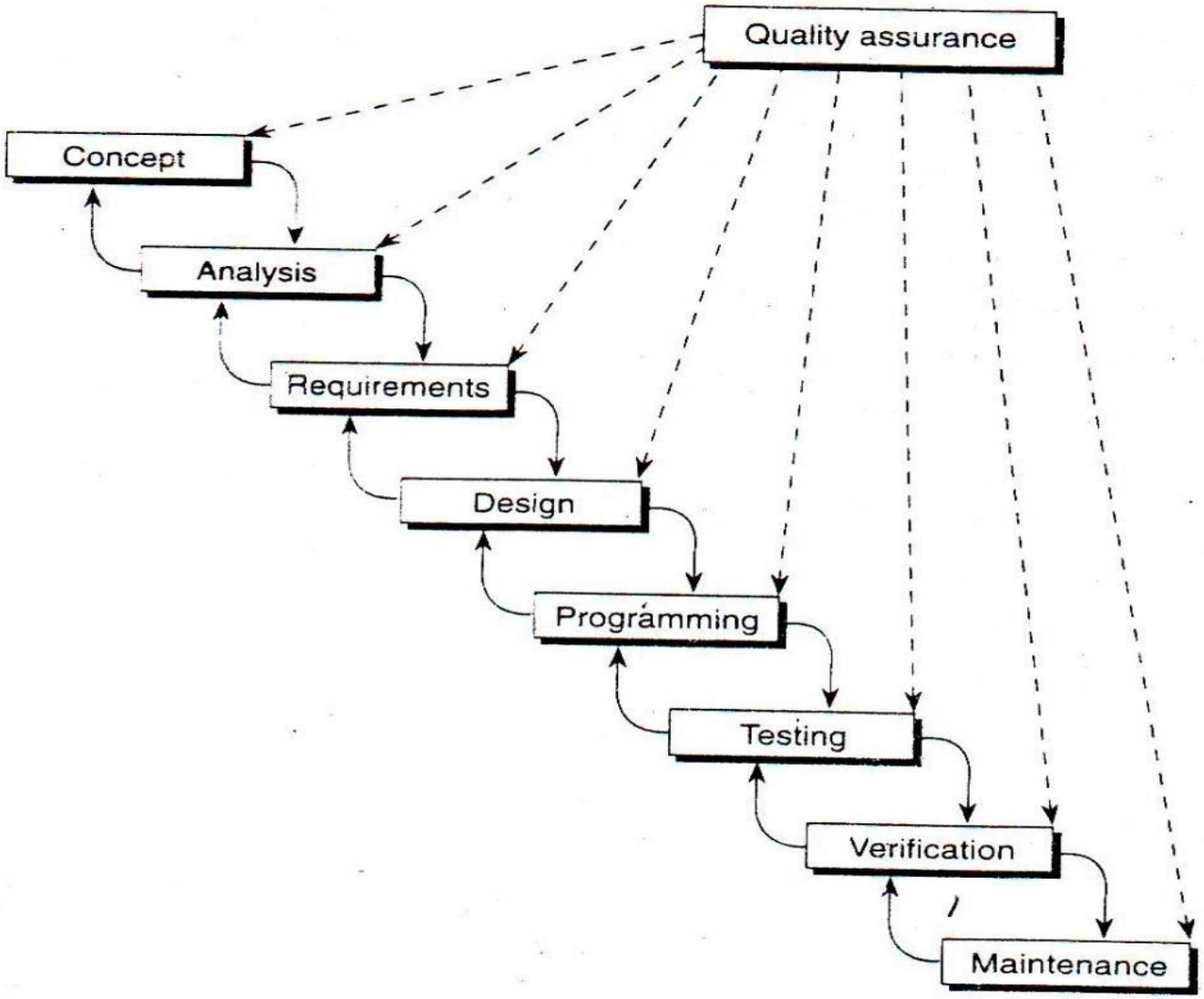


FIG. 11.1 The waterfall model of software development.

Quality Assurance

- Oversees each steps of model towards producing useful, reliable software rather than connection of modules
- Methods are necessary but not sufficient to produce useful, reliable software.
- First: Classify your system and its software according to any relevant standards.
- Second: Software development plan:
  - Hazard and fault - tree analysis for life critical functions
  - Configuration management: ensures current and correct version is released.
  - Documentation
  - Traceability

Concept & Analysis:

- Problem described in human language.
- Model the concept with mathematical formulas and algorithms.
- Analyze the software within the framework of the system description and concept.
- Analyse on the interactions and interfaces between the software, the hardware and data inputs.
- Analyze should concentrate on where most problem occurs which include:
  - technical tradeoff
  - performance timing
  - human factor
  - hazard and risk analysis
  - Fault tree analysis: Graphical model of sequential and concurrent events that leads to failure of problems

Requirements

- Reduce the abstract intentions of the customer into realizable constraints.
- Tells what the software does
- Includes standards that the software must adhere to, the development process, the constraints, and reliability or fault tolerance
- Requirements may call for several, successive specifications:
  - General specification
  - Functional performance specification
  - Requirements specification
  - Design specification
- Needs constraints considerations such as memory, timing margin, hardware, communication, I/O and execution speed;
- Will the selected processor and associated hardware support the requirements?
- Can the software use these resources and satisfy the requirements?

Design

- Design tells how the software does its functioning.
- Specifies how the software will fulfill the requirements.
- Consider these software elements in preparing the design:
  - System preparation and setup
  - Operating system and procedures
  - Communication and I/O
  - Monitoring procedures
  - Fault recovery and special procedures
  - Diagnostics features
- Interfaces demands most attention communication format:
  - Polled I/O
  - Interrupt I/O
  - Synchronization between tasks
  - Intertask signaling
  - Communications of polling and queuing to avoid overrunning events
- Design can specify algorithms and techniques that optimize performance, management of memory.
- Design may reuse modules and libraries in an effort to improve productivity and reliability.

Programming

- The methods of programming – assembly language, high level languages.
- CASE (Computer Added Software Engineering) tools are on the horizon
- Tools, language, methods
- Source file (Assembly language mnemonics) - **assembler - object file – linker – Binary** machine code – **Burn - PROM**

Testing

A) Internal reviews

- By colleagues examine the correctness of software and can figure out mistakes and error in logic
- More than 50% of errors, can be found and correct by code inspection or audit

B) Black box testing:

- To ensure that input and output interfaces are functioning correctly without concerning what happens in software.
- Ensures the integrity of the information flow.

C) White box testing:

- Exercises all logical decisions and functional path within a software module.
- Requires intimate knowledge of software module
- Useful for determining bugs on special case
- Exhaustive testing is impossible; may take 100 of years to test each and every possible combination

D) Alpha and Beta testing:

- Type of black box testing in actual environment
- **Alpha testing - programmer collaborate with user**
- **Beta testing** - user isolated from programmer

Verification

- Debuggers, logic analyzer, in circuit analyzer, in circuit debugger etc.

Maintenance:

- Require to control the software configurations: reports, measurement, personnel costs and documentation
- Plans for releasing software upgrades, to achieve consistency and continuity of the product.
- Cannot separate software maintenance from system concern

Disadvantages of Waterfall Model:

- Traditional view of software development
- Develop each component sequentially
- Not iterative - difficult to climb up the waterfall
- Focused on software rather than work design
- One phase is completed, documented and signed off before next phase for quality assurance
- Difficult to respond to changing customer requirement
- Software only available at late development schedule
- Based on hardware engineering model widely used in defence/aerospace
- Waterfall develops each component sequentially and usually does not iterate through more than a stage. In reality software seldom develops according to that model.

Benefits of Waterfall Model

- Managers *love waterfall models*
- Minimizes change, maximizes predictability
- Costs and risks are more predictable
- Highly documented
- Can be used for the projects whose requirements will not changeable
- Each stage has milestones and deliverables: project managers can use to gauge how close project is to completion
- Sets up division of labor: many software shops associate different people with different stages:
  - Systems analyst does analysis,
  - Architect does design,
  - Programmers code,
  - Testers validate, etc.

Problems with Waterfall Model

- Offers no insight into how each activity transforms artifacts (documents) of one stage into another
- Fails to treat software a problem-solving process
  - Unlike hardware, software development is not a manufacturing but a creative process
  - Manufacturing processes really can be linear sequences, but creative processes usually involve back-and-forth activities such as revisions
  - Software development involves a lot of communication between various human stakeholders
- Complex documentation requires highly profiled manpower
- Cannot be used for changing requirements
- Nevertheless, more complex models often embellish the waterfall,
  - incorporating feedback loops and additional activities

2. Prototyping Model

- In this model the developer and client interact to establish the requirements of the software.
- Accommodates the problem of changing requirements and make a subset of the software available early.
- Essence of prototyping is a quickly designed model that can undergo immediate evaluation.
- Define the broad set of objectives.
- This is follow up by the quick design, in which the visible elements of the software, the input and the output are designed.
- The quick design stresses the client's view of the software.
- The final product of the design is a prototype.
- The client then evaluates the prototype and provides its recommendations and suggestion to the analyst.

- The process continues in an iterative manner until the all the user requirements are met.
- Accommodates problem of changing requirements
- Helps to identify missing client requirements
- A prototype may take one of three forms
  - A paper model or computer based simulation
  - A program with a subset of functions
  - An existing program with other features that will be modified for your product.

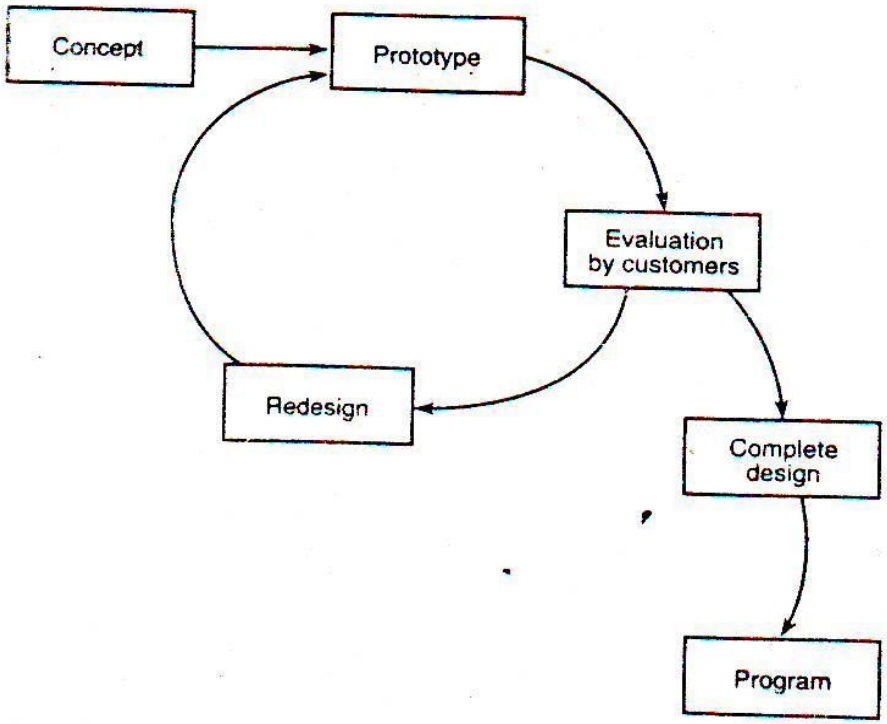


FIG. 11.4 Prototyping model for software development.

Advantages of Prototyping Model

- Due the interaction between the client and developer right from the beginning, the objectives and requirements of the software is well established.
- Suitable for the projects when client has not clear idea about his requirements.
- The client can provide its input during development of the prototype.
- The prototype serves as an aid for the development of the final product.

Disadvantages of Prototyping Model

- The quality of the software development is compromised in the rush to present a working version of the software to the client.
- The clients looks at the working version of the product at the outset and expect the final version of the product to be deliver immediately. This cause additional pressure over the developers to adopt shortcut in order to meet the final product deadline.
- It becomes difficult for the developer to convince the client as why the prototype has to be discarded.
- Sometimes prototype ends as final product which result in quality + maintenance problem
- Client may divert attention solely to interface issue
- Testing + documentation forgotten
- Designer tends to rush product to market without considering long term reliability, maintenance, configuration control
- Creeping featurism: The customer voices new desires after each evolution, and the project effort balloons.

3. Spiral Model

- Uses incremental approach to development that provides a combination of waterfall and prototyping model.
- Each cycle around the development spiral provides a successively more complete version of the software.
- Model allows flexibility to manage requirements control changes
- Used in proprietary application
- Spiral Model – risk driven rather than document driven
- The "risk" inherent in an activity is a measure of the uncertainty of the outcome of that activity
- High-risk activities cause schedule and cost overruns
- Risk is related to the amount and quality of available information. The less information, the higher the risk

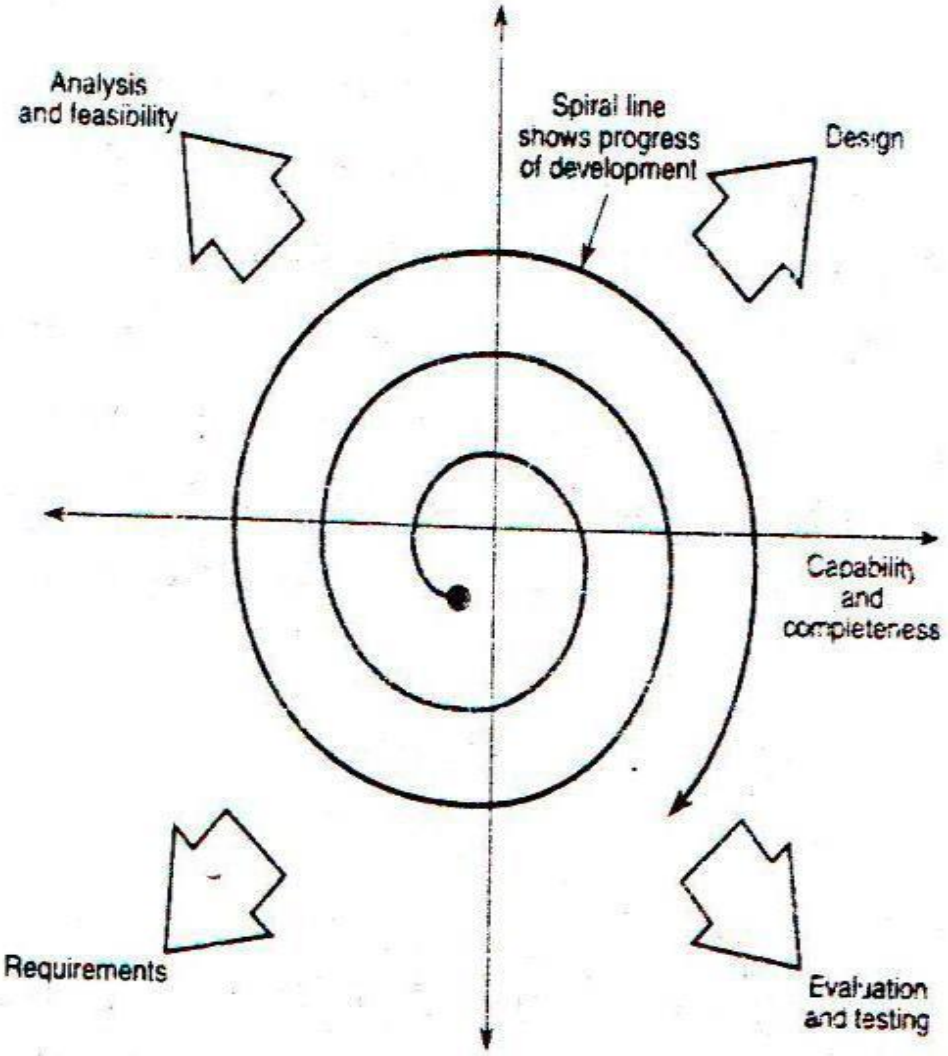


FIG. 11.5 Spiral model of software development.

Spiral Model Strengths

- Introduces risk management
- Prototyping controls costs
- Evolutionary development
- Release builds for beta testing
- Marketing advantage

Spiral Model Weaknesses

- Lack of risk management experience
- Lack of milestones
- Management is dubious of spiral process
- Change in Management
- Prototype Vs Production

Agile Model / Method

- Agile is a type of software development methodology that anticipates the need for flexibility and applies a level of pragmatism to the delivery of the finished product.
- Agile software development requires a cultural shift in many companies because it focuses on the clean delivery of individual pieces or parts of the software and not on the entire application.
- Benefits of Agile include its ability to help teams in an evolving landscape while maintaining a focus on the efficient delivery of business value.
- The collaborative culture facilitated by Agile also improves efficiency throughout the organization, as teams work together and understand their specific roles in the process.
- Finally, companies using Agile software development can feel confident that they're releasing a high-quality product because testing is performed throughout development.
- This provides the opportunity to make changes as needed and alert teams to any potential issues.

- **Agile software development** is an iterative, collaborative approach to building software that emphasizes flexibility, customer feedback, and continuous improvement.
- It’s centered around delivering working software in small, usable increments.
- The **Agile model** is a **flexible and iterative** approach to software development that focuses on **customer collaboration, continuous improvement**, and **rapid delivery of functional software**.
- Unlike the traditional **Waterfall model**, Agile doesn’t require all requirements to be defined at the start.
- Instead, it adapts throughout the project.

Component	Description
Iterations (Sprints)	Short, fixed-length cycles (usually 1–4 weeks) to develop and deliver features.
Incremental Delivery	Working software is delivered at the end of each sprint.
User Stories	Requirements are captured as short descriptions from the user's point of view.
Backlog	A prioritized list of tasks/features to be implemented. Managed by the Product Owner.
Team Collaboration	Developers, testers, and stakeholders work closely and communicate daily.
Customer Feedback	Customer reviews the software at the end of each iteration and gives feedback.
Continuous Testing & Integration	Code is tested and integrated frequently to catch issues early.
Retrospectives	Regular team meetings to reflect and improve the development process.

Agile Software Development Lifecycle

Requirement Gatheringrequirements
•Gather high-level business in the form of **epics** and **user stories**.

Planning
•Break down stories into **sprints** and **tasks**.
•Estimate effort and define the **definition of done (DoD)**.

Design
•High-level design is done for current sprint only.
•Encourages **evolutionary design** during the project.

Development
•Code is written during the sprint.
•Often uses **Test-Driven Development (TDD)** and **Pair Programming**.

Testing

- Testing is **continuous and automated**.
- Includes unit tests, integration tests, and acceptance tests.

Deployment

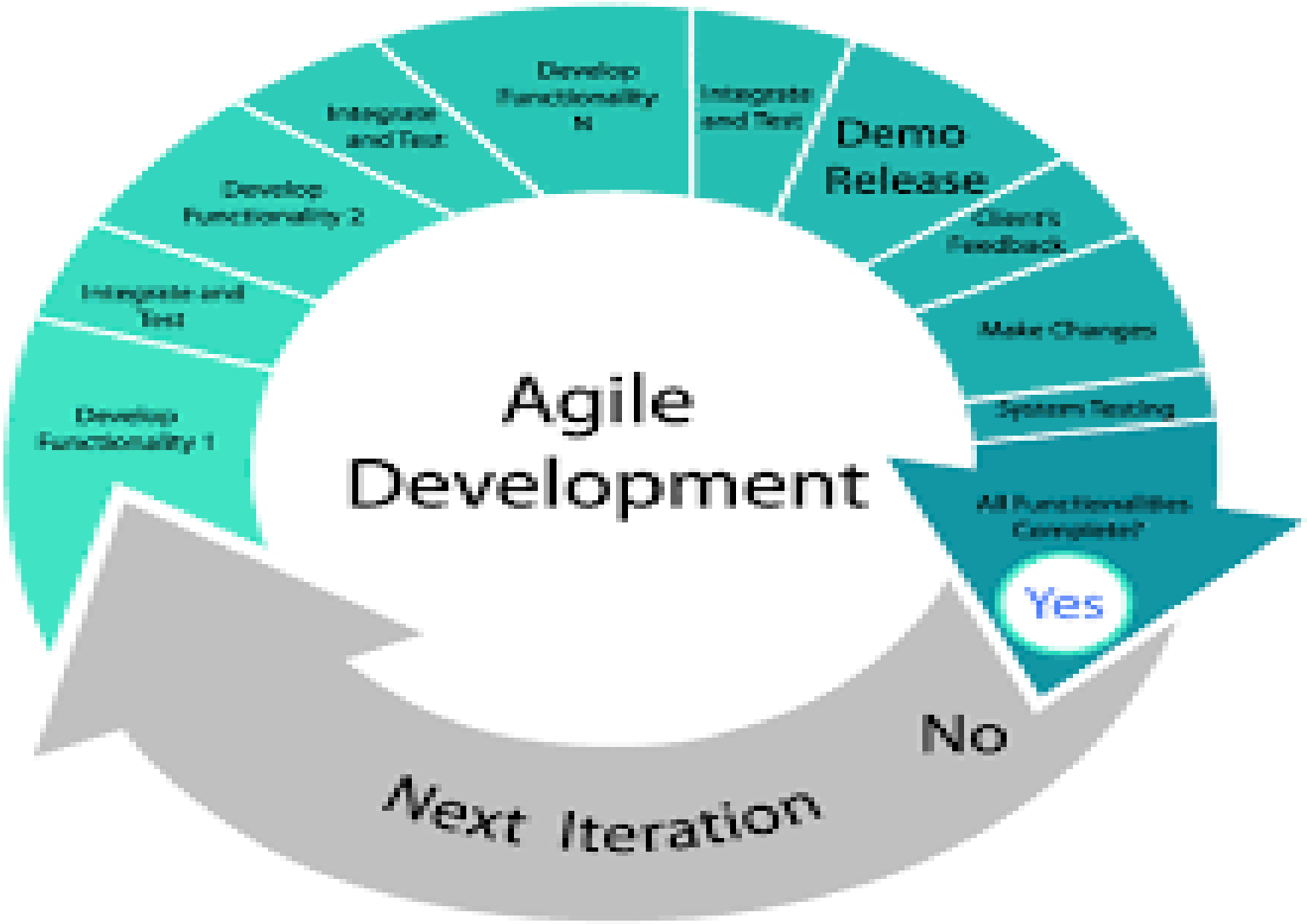
- Working software is delivered at the end of each sprint.
- Can involve **Continuous Deployment (CD)** pipelines.

Review and Feedback

- Stakeholders review the product increment.
- Suggestions are added to the backlog for future sprints.

Retrospective

- Team discusses what went well, what didn't, and plans improvements for the next sprint.


Develop Functionality 1
Integrate and Test
Develop Functionality 2
Integrate and Test
Develop Functionality N
Integrate and Test
Demo Release
Client's Feedback
Make Changes
System Testing
All Functionalities Complete?
Yes
No
Next Iteration
Agile Development

Metrics

- Objective understanding of the completion of the software at each stage or of its usefulness
- Software size, development time, personnel requirements, productivity, and number of defects all interrelate metric can define those relationships
- Cost and schedule are very poor metrics for producing quality software
- To rely on your estimates, you need to track the metrics honestly and record them carefully & consistently
- Good process model needs metrics to access performance and progress of software
  - Correctness, Reliability, Efficiency, Maintainability, Flexibility, Testability, Portability, Reuse, Utility, Size etc.

Process

- Process incorporate models of software development to generate useful, reliable and maintainable software
- Good process keeps statistics for feedback & improvement of the software development
- Software tools are integral to the software process; don’t change them in midstream.
- Process maturity levels defined by the software engineering institute:
  - Initial: Chaos or ad hoc process
  - Repeatable: Design & Management defined
  - Defined: Fully defined & enforced technical practices
  - Managed Process: Feedback that detects and prevents problems, a control process
  - Optimizing Process: Automating, monitoring & introducing new technologies

Software Limitations

- Not all problems can be solved
- Specifications cannot anticipate all possible uses and problems
- Errors creep into development in a number of ways
- Software simulation can predict only known outcomes
- Human error can occur in operating the software

Software Reliability

- Reliability is a broad concept.
  - It is applied whenever we expect something to behave in a certain way.
- Reliability is one of the metrics that are used to measure quality.
- It is a user-oriented quality factor relating to system operation.
  - Intuitively, if the users of a system rarely experience failure, the system is considered to be more reliable than one that fails more often.
- A system without faults is considered to be highly reliable.
  - Constructing a correct system is a difficult task.
  - Even an incorrect system may be considered to be reliable if the frequency of failure is “acceptable.”

- Key concepts in discussing reliability:
  - Fault
  - Failure
  - Time
- Reliability develops complete plan
- Understands nature of bugs, their introduction and removal.

Guidelines to write reliable software

- Make each module independent
- Reduce the complexity of each task
- Isolate tasks from influences, both hardware and timing
- Communicate through a single well-defined interface between tasks

Fault Tolerance

- Fault tolerance concerns safety and operational uptime, not reliability.
- It defines how a system prevents or responds to bugs, errors, faults or failures.
- Use
  - Check sums on blocks of memory to detect bit flips
  - Watchdog timer: h/w that monitors a system characteristic to check the control flow and signals the processor with logic pulse when it detect fault.
  - Roll-Back-Recovery or Roll-Forward-Recovery
  - Careful design
  - Redundant architecture

Software Bugs and Testing

Phases of bugs

- Intent
- Translation
- Execution
- Operation

Intent

- Wrong assumption +misunderstanding
- Correctly solving wrong problems
- Viruses
- Slang-limits of operation to broadly or too narrowly defined

Translation

- Incorrect algorithm
- Incorrect analysis
- Misinterpretation

Execution

- Semantic error –does not know how command works
- Syntax error- rules of language
- Logic error- using wrong decision
- Range error-overflow /underflow error
- truncation error- incorrect rounding
- Data error –not initialing values, wrong error etc
- Language misuse-inefficient coding
- Documentation-wrong/misleading comments

Operation

- Changing paradigm
- Interface error
- Performance
- Hardware error
- Human error

Debugging

- Print statement
- Break points and watch values
- In circuit emulators ,in circuit debugger
- Logic analyzer
- White box testing
- Black box testing
- Grey Box testing
  - Having knowledge of internal data structure & algorithm for purpose of designing test case but testing is black box
  - Used in reverse engineering to determine instance, boundary value

Testing Levels

- Unit test
  - To test functionality of specific section of code at functional level
  - Building blocks work independently of each other
  - E.g. Class level testing in OOP
- Integration test
  - To verify interface between components
- System test
  - To test the whole system which is to be used